Universidad Nacional Autónoma de México
Facultad de Ingeniería

Sistemas Operativos
Grupo 7 Semestre 2027-1

Sistemas operativos en misiones espaciales

Qué hace el sistema operativo cuando no puede fallar

Daniel Berumen Valdez

17 de octubre de 2026

Índice

1. Introducción **3**

2. Conceptos fundamentales **4**

2.1. Sistemas de tiempo real 4

2.2. Planificación por prioridades con apropiación 5

2.3. Inversión de prioridades 5

2.4. Efectos de la radiación en la memoria 7

2.5. El *watchdog* y la recuperación por reinicio 7

3. Sistemas operativos y *frameworks* en misiones espaciales **8**

3.1. ¿Por qué no usar un sistema operativo común? 8

3.2. VxWorks 9

3.3. RTEMS 9

3.4. core Flight System (cFS) 10

3.5. F' (F Prime) 11

3.6. Comparación y relación con el curso 12

4. Casos de estudio **12**

4.1. Apolo 11 (1969): el sistema que se salvó a sí mismo 13

4.1.1. La computadora 13

4.1.2. El sistema operativo de la AGC 13

4.1.3. Las alarmas 1201 y 1202 14

4.1.4. La recuperación: reiniciar sin perder el rumbo 15

4.1.5. La decisión en Houston 15

4.1.6. Mitos y precisiones 15

4.1.7. Relación con el curso 16

4.2. Mars Pathfinder (1997): la inversión de prioridades llega a Marte 16

4.2.1. Contexto 16

4.2.2. Las tareas involucradas 17

4.2.3. Qué pasó 17

4.2.4. El reinicio como protección 18

4.2.5. El diagnóstico 18

4.2.6. La corrección 19

4.2.7. Más allá de la versión de los libros 19

4.2.8. El debate: ¿herencia o techo de prioridad? 19

4.2.9. Relación con el curso 20
4.3. Radiación: lo que hace el software cuando el hardware no alcanza 21
4.3.1. Qué le corresponde al hardware y qué al software 21
4.3.2. CALIPSO: el barrido de memoria como tarea del sistema 21
4.3.3. ARGOS: proteger el propio programa 23
4.3.4. Otras técnicas de software 23
4.3.5. Relación con el curso 24
4.4. Spirit (2004): cuando reiniciar no basta 24
4.4.1. Contexto 24
4.4.2. La causa: un recurso que se agotaba 24
4.4.3. El ciclo de reinicios 25
4.4.4. La recuperación: el modo *crippled* 26
4.4.5. Lecciones 26
4.4.6. Relación con el curso 27
4.5. Ingenuity (2021): Linux en Marte 27
4.5.1. Contexto 27
4.5.2. Una arquitectura dividida por niveles de criticidad 27
4.5.3. Cómo se configuró Linux 28
4.5.4. Incidente 1: el *watchdog* antes del primer vuelo 29
4.5.5. Incidente 2: el vuelo 6 y las marcas de tiempo 29
4.5.6. Relación con el curso 30

5. Discusión **31**
5.1. Resumen de los casos 31
5.2. Patrones comunes 31
5.2.1. Un vigilante y un reinicio controlado 31
5.2.2. Reiniciar solo sirve si elimina la causa 31
5.2.3. Separar lo crítico de lo que no lo es 32
5.2.4. Diagnóstico por diseño 32
5.2.5. Entender el software que se usa 32
5.3. ¿Sistema operativo de propósito general o de tiempo real? 32
5.4. Qué le corresponde al hardware y qué al sistema operativo 33
5.5. Limitaciones de este trabajo 34

6. Conclusiones **34**

Glosario **35**

1. Introducción

Cuando una computadora personal deja de responder, la solución suele ser sencilla: cerramos el programa o la reiniciamos. Lo peor que puede pasar es perder unos minutos de trabajo. Ahora imaginemos que esa computadora está en Marte, a millones de kilómetros de distancia, y que de ella depende una misión que tomó años construir. Nadie puede apretar el botón de reinicio, y cualquier orden enviada desde la Tierra tarda varios minutos en llegar.

En un entorno así, el sistema operativo carga con una responsabilidad mucho mayor. Como hemos visto en clase, el sistema operativo es el programa que administra los recursos de la computadora y coordina a los procesos que compiten por ellos. En una nave espacial, además, debe cumplir con algo que una computadora común no necesita garantizar: *responder a tiempo*.

A los sistemas con esta exigencia se les llama *sistemas de tiempo real*. Un sistema de tiempo real no es necesariamente un sistema rápido, sino uno en el que cada tarea tiene un plazo (*deadline*) que debe cumplirse. Si el cálculo que corrige la posición de una nave llega tarde, el resultado ya no sirve, aunque sea correcto. En estos sistemas, llegar tarde equivale a equivocarse.

El espacio agrega un segundo problema: la radiación. Las partículas de alta energía pueden cambiar el valor de un bit en la memoria o en el procesador, un fenómeno conocido como *bit flip*. El hardware de las naves incluye protecciones contra esto, pero ninguna es perfecta. Por eso el software también debe estar preparado para detectar errores y recuperarse de ellos.

La historia de la exploración espacial ofrece varios ejemplos de cómo el sistema operativo enfrentó estos retos. Durante el alunizaje del Apolo 11, en 1969, la computadora de guía mostró alarmas por sobrecarga, pero su software estaba diseñado para descartar el trabajo menos importante y seguir con el esencial (Mindell, 2008). En 1997, el Mars Pathfinder comenzó a reiniciarse repetidamente en la superficie de Marte por un problema de sincronización entre procesos, que los ingenieros resolvieron a distancia. Y en 2021, el helicóptero Ingenuity se convirtió en la primera aeronave en volar en otro planeta usando Linux, un sistema operativo de propósito general, como parte de su arquitectura (Balaram et al., 2018).

En todos estos casos aparece la misma idea: el sistema operativo no solo debe funcionar bien, sino también *fallar bien*. Es decir, cuando algo sale mal, debe hacerlo de una forma controlada que permita recuperar la misión. De ahí surge la pregunta que guía este trabajo:

> *¿Qué hace el sistema operativo de una nave espacial cuando no puede fallar, y qué hace cuando falla de todos modos?*

Es importante aclarar el alcance del documento. Muchas de las defensas de una nave dependen del hardware, como la memoria con corrección de errores, el blindaje contra radiación

o los procesadores duplicados. Estos elementos se mencionarán solo como contexto, para distinguir qué le corresponde al hardware y qué al sistema operativo. El enfoque principal estará en las tareas del sistema operativo y del software de vuelo: la planificación de procesos, la sincronización, la vigilancia del sistema y la recuperación ante fallas.

El documento se organiza de la siguiente manera. La sección 2 presenta los conceptos necesarios para entender los casos: tiempo real, planificación por prioridades, inversión de prioridades, efectos de la radiación y mecanismos de recuperación. La sección 3 describe los sistemas operativos y *frameworks* que se usan en misiones espaciales. La sección 4 analiza cinco casos de estudio, desde el Apolo 11 hasta Ingenuity. Finalmente, la sección 5 discute los patrones comunes entre los casos y la sección 6 presenta las conclusiones.

2. Conceptos fundamentales

Esta sección presenta las ideas necesarias para entender los casos de estudio. Algunas de ellas, como la planificación de procesos, se estudian más adelante en el curso, por lo que se explican aquí de forma introductoria.

2.1. Sistemas de tiempo real

En una computadora de escritorio, lo importante es que los programas den el resultado correcto. Si un programa tarda un segundo más de lo normal, el usuario apenas lo nota. En un sistema de tiempo real, en cambio, el resultado debe ser correcto *y además* llegar antes de un plazo (*deadline*).

Se distinguen dos tipos de sistemas de tiempo real:

- **Tiempo real duro** (*hard real-time*): incumplir un plazo, aunque sea una vez, puede provocar una falla grave. Un ejemplo es la bolsa de aire de un automóvil: si se infla medio segundo tarde, ya no protege a nadie.
- **Tiempo real suave** (*soft real-time*): incumplir un plazo de vez en cuando empeora el resultado, pero no lo arruina. Un ejemplo es una videollamada: si un cuadro llega tarde, la imagen se congela un instante, pero la llamada continúa.

Las naves espaciales combinan ambos tipos. Controlar la orientación de una nave o la estabilidad de un helicóptero es tiempo real duro. Enviar fotografías a la Tierra o guardar registros es tiempo real suave. Como se verá en los casos de estudio, separar bien estas dos clases de tareas es una de las decisiones de diseño más importantes.

2.2. Planificación por prioridades con apropiación

Cuando varios procesos o hilos están listos para ejecutarse, el sistema operativo debe decidir cuál usa el procesador. A esta decisión se le llama *planificación*, y la parte del sistema operativo que la toma es el *planificador* (*scheduler*).

Los sistemas de tiempo real suelen usar un esquema sencillo: cada tarea recibe una *prioridad* según qué tan importante o urgente es, y el planificador siempre ejecuta la tarea lista de mayor prioridad. Además, este esquema es *con apropiación* (*preemptive*): si una tarea de alta prioridad se vuelve lista mientras corre una de menor prioridad, el sistema operativo interrumpe a la segunda y le entrega el procesador a la primera de inmediato.

La ventaja de este esquema es que resulta predecible: las tareas más críticas siempre se atienden primero, sin esperar a que las demás terminen. Sin embargo, como muestra la siguiente sección, esta garantía puede romperse cuando las tareas comparten recursos.

2.3. Inversión de prioridades

En clase hemos visto que, cuando varios procesos comparten un recurso, se usa un mutex para garantizar la exclusión mutua: solo quien tiene el mutex puede entrar a la sección crítica, y los demás deben esperar. El problema aparece cuando combinamos el mutex con la planificación por prioridades.

Consideremos tres tareas con prioridades distintas: una de prioridad alta (A), una de prioridad media (M) y una de prioridad baja (B). Las tareas A y B comparten un recurso protegido por un mutex; la tarea M no lo usa. La secuencia problemática es la siguiente:

1. B toma el mutex y entra a su sección crítica.
2. A se vuelve lista. Por tener mayor prioridad, desplaza a B.
3. A intenta tomar el mutex, pero B lo tiene. A se bloquea.
4. Antes de que B termine su sección crítica, M se vuelve lista. Como M tiene más prioridad que B, el planificador ejecuta a M.
5. Mientras M siga trabajando, B no avanza, no libera el mutex y A sigue bloqueada.

El resultado es que la tarea de mayor prioridad termina esperando a una de prioridad media que ni siquiera usa el recurso compartido. A esta situación se le llama *inversión de prioridades* (*priority inversion*). Que A espere a B mientras B termina su sección crítica es normal y tiene un límite. Lo grave es que la espera se alargue por culpa de M, porque no hay forma de saber cuánto durará. A esto se le conoce como inversión de prioridades *no acotada*.

El problema ya había sido descrito a finales de la década de 1970 por los diseñadores del lenguaje Mesa (Lampson & Redell, 1980).

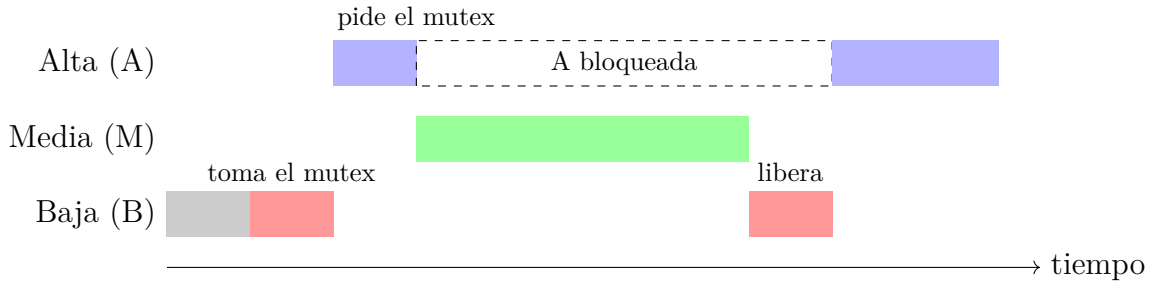


Figura 1: Inversión de prioridades. En rojo, la sección crítica de B. La tarea M retrasa a B y, con ello, a la tarea A, que tiene la mayor prioridad.

Existen dos soluciones clásicas, formalizadas por Sha et al. (1990):

- **Herencia de prioridad** (*priority inheritance*): mientras B tenga un mutex que A necesita, B «hereda» temporalmente la prioridad de A. Así, M ya no puede interrumpirla, B termina pronto su sección crítica, libera el mutex y recupera su prioridad original.
- **Techo de prioridad** (*priority ceiling*): a cada mutex se le asigna de antemano la prioridad de la tarea más importante que pueda usarlo. Este protocolo es más estricto: garantiza que una tarea de alta prioridad espere, como máximo, a que termine una sola sección crítica de menor prioridad, y además evita los bloqueos mutuos (*deadlocks*).

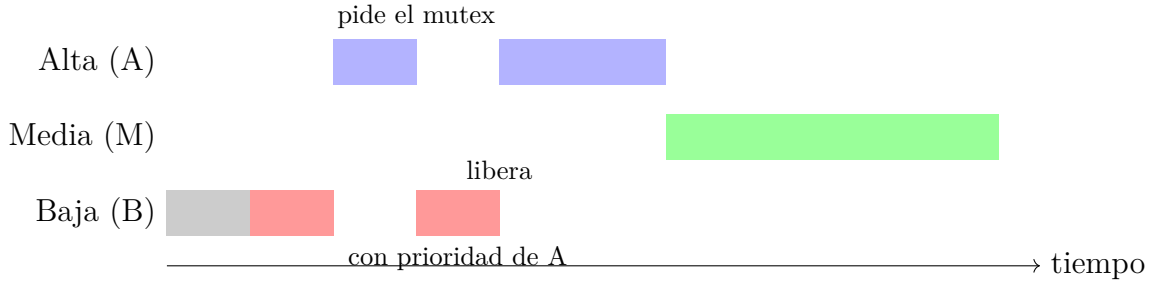


Figura 2: Misma situación con herencia de prioridad. B termina su sección crítica sin que M la interrumpa, y A continúa mucho antes.

Como se verá en el caso del Mars Pathfinder, la elección entre estas soluciones no es solo teórica: decidir si un mutex usa o no herencia de prioridad puede determinar el éxito de una misión.

2.4. Efectos de la radiación en la memoria

En el espacio, la computadora de una nave recibe partículas de alta energía provenientes del Sol y de otras fuentes. Cuando una de estas partículas golpea un circuito de memoria, puede cambiar un bit de 0 a 1 o de 1 a 0. A este evento se le llama *Single Event Upset* (SEU), o simplemente *bit flip*. No daña el hardware de forma permanente, pero sí altera la información: un dato, una instrucción del programa o una variable del sistema operativo pueden cambiar sin que nadie lo haya ordenado.

La primera línea de defensa es el hardware. Las memorias con *código de corrección de errores* (*Error Correcting Code*, ECC) guardan bits adicionales que permiten detectar y corregir un bit alterado al momento de leerlo. Sin embargo, esta protección tiene límites. En muchos sistemas, el ECC corrige el valor que entrega al procesador, pero no reescribe la memoria. Si un dato no se lee durante mucho tiempo, pueden acumularse dos o más errores en la misma palabra, y entonces el ECC ya no alcanza a corregirlos.

Aquí interviene el software. Una técnica común es el *barrido de memoria* (*memory scrubbing*): una tarea del sistema recorre periódicamente toda la memoria, lee cada dato para que el ECC lo corrija y lo vuelve a escribir limpio. Así los errores se eliminan antes de acumularse. Otras técnicas protegen al propio programa, por ejemplo guardando copias redundantes del código y verificándolas en software, lo que permite operar con éxito incluso en procesadores que no fueron diseñados para el espacio (Shirvani et al., 2000).

2.5. El *watchdog* y la recuperación por reinicio

Algunos trenes tienen un mecanismo llamado «pedal del hombre muerto»: el conductor debe mantenerlo presionado o activarlo cada cierto tiempo, y si deja de hacerlo, el tren frena automáticamente. La idea es que, si el conductor sufre un desmayo, el sistema no siga avanzando sin control.

Las computadoras de las naves usan un mecanismo equivalente llamado *watchdog* («perro guardián»). Se trata de un temporizador que el software debe reiniciar periódicamente. Si el temporizador llega a cero porque el software se trabó, entró en un ciclo infinito o una tarea dejó de responder, el *watchdog* concluye que algo salió mal y reinicia el sistema o el componente afectado.

Reiniciar puede parecer una medida drástica, pero en el espacio suele ser la más segura: es preferible volver a un estado conocido y confiable que seguir operando en un estado desconocido. Por eso, los sistemas de vuelo se diseñan para que reiniciarse sea una operación rápida y segura, que conserve los datos importantes.

Una idea relacionada es dividir el software en *componentes aislados*, de modo que, si uno

falla, pueda reiniciarse sin afectar al resto. Por ejemplo, en el *framework* F' de la NASA, un componente llamado *Health* envía periódicamente una señal de verificación (*ping*) a cada componente activo. Si alguno no responde a tiempo, se reporta una falla, y el *watchdog* deja de ser atendido (NASA Jet Propulsion Laboratory, s.f.). De este modo, un solo componente trabado basta para que el sistema reaccione.

Estos cuatro conceptos (tiempo real, planificación por prioridades, inversión de prioridades y recuperación ante fallas) aparecerán una y otra vez en los casos de estudio. Antes de analizarlos, la siguiente sección describe los sistemas operativos que se usan en las misiones espaciales.

3. Sistemas operativos y *frameworks* en misiones espaciales

Antes de analizar los casos de estudio, conviene conocer el software sobre el que funcionan las naves. En la mayoría de las misiones se combinan dos piezas: un *sistema operativo de tiempo real* (*Real-Time Operating System*, RTOS), que administra el procesador y los procesos, y un *framework* de software de vuelo, que organiza los programas de la misión encima de ese sistema operativo.

3.1. ¿Por qué no usar un sistema operativo común?

Un sistema operativo de escritorio funciona como un restaurante: intenta atender bien a todos los clientes *en promedio*, aunque a veces alguno espere más de la cuenta. Un RTOS, en cambio, funciona como una sala de urgencias: el paciente más grave entra primero y se sabe cuánto tiempo, como máximo, tardará en ser atendido.

La documentación del core Flight System de la NASA resume lo que se le pide al sistema operativo de una nave: planificación por prioridades con apropiación, colas de mensajes, semáforos, manejo de interrupciones y sistemas de archivos. En esa misma documentación, la NASA aclara que Linux no es un sistema de tiempo real y que solo lo usa para desarrollar y probar en computadoras de escritorio (McComas, 2012).

Además, el hardware de las naves es muy distinto al de una computadora común. Los procesadores se fabrican especialmente para resistir la radiación, lo que los hace mucho más lentos que los comerciales. Por ejemplo, el procesador RAD750, usado en los rovers Curiosity y Perseverance, trabajaba originalmente a 133 MHz, pero a cambio presenta apenas un error por radiación cada 10^{11} bits por día (Berger et al., 2001). Esto obliga a que el sistema operativo sea pequeño y eficiente.

Un detalle que sorprende es que muchos RTOS espaciales no separan el espacio de usuario del espacio del núcleo, como sí lo hacen los sistemas operativos que estudiamos en clase. En RTEMS, por ejemplo, el sistema operativo se compila junto con la aplicación como si fuera una biblioteca, y todos comparten la misma memoria sin protección entre ellos (European Space Agency, 2026). Se sacrifica protección a cambio de velocidad y sencillez, y por eso el software se revisa y prueba con mucho más rigor.

3.2. VxWorks

VxWorks es un RTOS comercial desarrollado por la empresa Wind River. Su núcleo usa planificación por prioridades con apropiación y maneja 256 niveles de prioridad, del 0 al 255, donde el 0 es la prioridad más alta (Wind River Systems, 2002).

VxWorks es el sistema operativo de todos los rovers que la NASA ha enviado a Marte. Para el Mars Pathfinder (1997), Wind River adaptó VxWorks a la arquitectura RS/6000 de IBM, cuya versión resistente a la radiación era el procesador de vuelo de la nave (G. E. Reeves, 1997). Desde entonces lo han usado los rovers Spirit y Opportunity (2004), Curiosity (2012) y Perseverance (2021). Este último tiene dos computadoras RAD750 idénticas, de modo que si una falla la otra puede tomar el control (Jet Propulsion Laboratory, 2021). Su software de vuelo está dividido en módulos, y cada uno corre como una tarea independiente de VxWorks (Rabideau et al., 2017).

3.3. RTEMS

RTEMS es un RTOS de código abierto. Su desarrollo comenzó en 1988 por encargo del Ejército de Estados Unidos, que buscaba un sistema operativo de tiempo real sin costos de licencia y con acceso completo al código fuente (Sherrill, 2016). Sus siglas originalmente significaban *Real-Time Executive for Missile Systems*; con el tiempo la «M» pasó a significar *Military* y después *Multiprocessor* (OAR Corporation, 2000). Su código se hizo abierto a principios de la década de 1990.

Hoy RTEMS es muy usado por la NASA y, sobre todo, por la Agencia Espacial Europea (ESA). Para facilitar su uso en misiones, la ESA financió un *paquete de datos de calificación* (*Qualification Data Pack*, QDP): un conjunto de documentos y pruebas automáticas que demuestran que RTEMS cumple las normas europeas de software espacial. Desde marzo de 2024, este paquete alcanza la categoría B de criticidad, para los procesadores europeos GR712RC y GR740 (European Space Agency, 2026). Conviene precisar que RTEMS está *precalificado*, no certificado: cada misión debe repetir las pruebas con su propio hardware y su propia aplicación.

Algunas misiones de la NASA que usan RTEMS son MMS, que estudia el campo magnético de la Tierra, y la sonda solar Parker (McComas, 2012).

3.4. core Flight System (cFS)

Durante años, cada misión del centro Goddard de la NASA copiaba el software de la anterior y lo modificaba a mano. Después de más de diez misiones así, los ingenieros decidieron crear un software reutilizable: el *core Flight System* (McComas, 2012).

cFS no es un sistema operativo, sino un *framework* que funciona encima de uno. Está organizado en capas, de modo que cada capa solo conoce a la que está debajo:

Capa	**Función**
Aplicaciones	Los programas propios de cada misión.
cFE	Servicios comunes a todas las aplicaciones.
OSAL y PSP	Ocultan las diferencias entre sistemas operativos y entre tarjetas de hardware.
RTOS	VxWorks, RTEMS o Linux.
Hardware	El procesador y los dispositivos de la nave.

Tabla 1: Capas del core Flight System, de arriba hacia abajo.

La capa más interesante desde el punto de vista del curso es el OSAL (*Operating System Abstraction Layer*). Ofrece una sola forma de pedir servicios al sistema operativo y la traduce a la llamada correspondiente de cada uno. Por ejemplo, para crear una tarea:

Sistema operativo	**Llamada real**
VxWorks	`taskSpawn`
RTEMS	`rtems_task_create` y `rtems_task_start`
Linux (POSIX)	`pthread_create`

Tabla 2: Traducción que hace el OSAL de la llamada `OS_TaskCreate`.

Gracias a esto, el mismo programa puede funcionar en distintos sistemas operativos sin modificarse.

Encima del OSAL está el cFE (*core Flight Executive*), que ofrece cinco servicios (McComas, 2012):

- **Executive Services**: arranca, reinicia y elimina aplicaciones, incluso mientras la nave está en operación.

- **Software Bus**: permite que las aplicaciones se comuniquen enviando mensajes, sin conocerse entre sí. Una aplicación *publica* un mensaje y las que estén interesadas se *suscriben* a él.
- **Event Services**: registra eventos importantes con la hora en que ocurrieron.
- **Table Services**: maneja tablas de parámetros que pueden actualizarse desde la Tierra.
- **Time Services**: mantiene la hora de la nave.

Para la pregunta de este trabajo, el primer servicio es el más importante. El cFE puede reiniciar una sola aplicación, o todo el procesador, y conserva los datos críticos en un área especial de memoria para recuperarlos después del reinicio. Además, una aplicación llamada *Health and Safety* vigila que ninguna aplicación se quede trabada y es la encargada de atender el *watchdog* (McComas, 2012).

El cFE voló por primera vez en la sonda Lunar Reconnaissance Orbiter (2009), y el conjunto inicial de aplicaciones de cFS se estrenó en el satélite GPM (2014). La NASA abrió el código del cFE en 2011 y el del sistema completo a inicios de 2015 (Core Flight System, s.f.).

3.5. F' (F Prime)

F' es un *framework* desarrollado por el Jet Propulsion Laboratory (JPL) para naves pequeñas e instrumentos. Igual que cFS, funciona encima de un sistema operativo, pero organiza el software de otra manera: en *componentes* conectados entre sí por *puertos*, como piezas que se enchufan (Bocchino et al., 2018).

Hay tres tipos de componentes:

- **Activos**: tienen su propio hilo y una cola donde reciben mensajes.
- **Pasivos**: no tienen hilo propio; se ejecutan en el hilo de quien los llama.
- **En cola** (*queued*): tienen cola, pero no hilo propio.

Lo interesante para el curso es cómo F' resuelve la concurrencia. Un puerto puede ser *guarded*, en cuyo caso toma un mutex antes de ejecutar y lo libera al terminar, o *asíncrono*, en cuyo caso solo deja un mensaje en la cola del componente. En este segundo caso no se necesita mutex, porque la cola ya garantiza que los mensajes se atiendan uno por uno (Bocchino et al., 2018). Son dos soluciones al mismo problema de exclusión mutua que estudiamos en clase.

F' también incluye *grupos de tasa* (*rate groups*), que despiertan a los componentes con una frecuencia fija, y el componente *Health*, descrito en la sección anterior, que vigila que todos sigan respondiendo.

F' voló en el instrumento ISS-RapidScat de la Estación Espacial Internacional, en el pequeño satélite ASTERIA y en el helicóptero Ingenuity, donde funcionó sobre Linux. El JPL lo liberó como código abierto en 2017 (NASA/JPL, 2021b).

3.6. Comparación y relación con el curso

Aspecto	**cFS**	**F'**
Creador	NASA Goddard	NASA JPL
Lenguaje	C	C++
Unidad básica	Aplicación (una tarea del RTOS)	Componente (activo, pasivo o en cola)
Comunicación	Bus de mensajes de publicar/suscribir	Puertos conectados directamente
Uso típico	Misiones grandes (LRO, GPM)	Naves pequeñas e instrumentos

Tabla 3: Diferencias principales entre cFS y F'.

Aunque ninguno de los dos es un sistema operativo, ambos aplican ideas que se estudian en el curso. El OSAL es un ejemplo de *capa de abstracción*, como la que separa a las aplicaciones del hardware en cualquier sistema operativo. Las aplicaciones de cFS y los componentes activos de F' son *hilos* que se comunican por *paso de mensajes*. Y la posibilidad de reiniciar una parte del sistema sin detener todo lo demás recuerda a la filosofía de los *microkernels*: servicios pequeños, aislados y reemplazables.

Con estas herramientas presentadas, la siguiente sección analiza los casos de estudio, donde se verá cómo funcionaron cuando algo salió mal.

4. Casos de estudio

Esta sección analiza cinco momentos de la exploración espacial en los que el sistema operativo enfrentó una situación crítica. Los casos están ordenados cronológicamente, y en cada uno se identifican los conceptos del curso que intervienen y la forma en que el sistema respondió a la falla.

4.1. Apolo 11 (1969): el sistema que se salvó a sí mismo

4.1.1. La computadora

El módulo lunar del Apolo 11 llevaba la *Apollo Guidance Computer* (AGC), diseñada por el Laboratorio de Instrumentación del MIT. Para los estándares actuales era diminuta: tenía apenas 2048 palabras de memoria borrable, que funcionaba como su RAM, y 36 864 palabras de memoria fija, de solo lectura, donde se guardaban los programas. Cada palabra tenía 15 bits de datos y uno de paridad para detectar errores (Eyles, 2004). Los programas se fabricaban literalmente tejiendo alambres a través de pequeños anillos magnéticos, por lo que una vez construida la memoria fija ya no podía modificarse.

Los astronautas se comunicaban con la computadora mediante el DSKY, un tablero con un teclado numérico y pequeñas pantallas. Las órdenes se daban con un código de «verbo» (qué hacer) y «sustantivo» (sobre qué dato hacerlo).

4.1.2. El sistema operativo de la AGC

Aunque en esa época no se usaba el término «sistema operativo» como hoy, la AGC tenía uno. Fue diseñado por J. Halcombe Laning a mediados de la década de 1960, sin ningún ejemplo previo en el cual basarse (Eyles, 2004). Tenía dos partes principales:

- **El *Executive***: administraba los *jobs*, que eran trabajos de cualquier duración, como calcular la posición de la nave. Cada *job* tenía una prioridad, y el *Executive* siempre elegía el de prioridad más alta.
- **El *Waitlist***: administraba las *tasks*, que eran rutinas muy cortas programadas para ejecutarse en un momento exacto. Las disparaba una interrupción del temporizador.

Ambas partes trabajaban juntas. Por ejemplo, cada dos segundos una *task* llamada READACCS leía los acelerómetros de la nave y luego registraba un *job* llamado SERVICER, encargado de procesar esos datos para calcular la navegación (Eyles, 2004).

Para registrar cada *job*, el *Executive* usaba un *core set*: un pequeño bloque de memoria donde guardaba su prioridad y su estado. Los *jobs* que hacían cálculos complejos necesitaban además un espacio adicional llamado *VAC area*. Había ocho *core sets*[1] y cinco *VAC areas* (Burkey, s.f.). Es decir, la AGC podía tener como máximo ocho trabajos registrados al mismo tiempo.

[1] Las fuentes no coinciden en este número: Eyles (2004) habla de ocho, mientras que Adler (1998) menciona siete. El código fuente original define un contador de siete, pero lo recorre de forma que se revisan ocho posiciones: la del *job* en ejecución y siete de reserva (Burkey, s.f.).

Un detalle interesante para el curso es la forma en que el *Executive* cambiaba de un *job* a otro. Aunque Eyles (2004) lo describe como apropiativo, el código muestra que el cambio era en realidad *cooperativo*: cuando llegaba un *job* más importante, el *Executive* solo lo marcaba como el siguiente en turno, y el *job* en ejecución cedía el procesador en puntos frecuentes de su código (Burkey, s.f.). La única apropiación real la hacían las interrupciones y las *tasks* del *Waitlist*.

4.1.3. Las alarmas 1201 y 1202

El 20 de julio de 1969, durante el descenso hacia la Luna, la computadora mostró cinco alarmas: cuatro con el código 1202 y una con el código 1201. La primera apareció unos siete minutos antes del aterrizaje (Mission Evaluation Team, 1969). Para entender qué pasó, hay que seguir la cadena de causas:

1. El *radar de encuentro*, que solo se necesitaba para volver a acoplarse con el módulo de mando, estaba encendido. Una razón era mantenerlo listo por si había que abortar el alunizaje.

2. Un documento de interfaz entre el sistema de guiado y la electrónica del radar pedía que dos señales eléctricas tuvieran la misma frecuencia, pero no exigía que estuvieran sincronizadas. Según cómo se encendiera el equipo, las señales podían quedar desfasadas.

3. Con ese desfase, el radar enviaba a la computadora miles de pulsos por segundo sin sentido. Cada pulso obligaba a la AGC a interrumpir lo que hacía para contarlo, lo que le robaba aproximadamente el 13 % de su tiempo de procesamiento (Eyles, 2004).

El problema era que la computadora ya trabajaba cerca de su límite. Durante el descenso solo le quedaba entre el 10 y el 15 % de tiempo libre. Con una pérdida del 13 %, simplemente no alcanzaba para todo (Eyles, 2004).

El *job* SERVICER tenía la prioridad más baja y era el más largo. Cuando READACCS lo volvía a registrar dos segundos después, la copia anterior todavía no había terminado y seguía ocupando su *core set* y su *VAC area*. Cada nueva copia ocupaba un espacio más, hasta que se agotaron. Cuando el *Executive* no encontraba un *core set* libre, mostraba la alarma 1202; cuando no encontraba una *VAC area*, mostraba la 1201 (Burkey, s.f.).

En términos del curso, es una situación de sobrecarga: la tabla de procesos se llenó porque el proceso de menor prioridad nunca alcanzaba a terminar.

4.1.4. La recuperación: reiniciar sin perder el rumbo

Lo notable del caso es lo que hizo el sistema después de la alarma. Alrededor de un año antes de la misión se había pedido que la computadora pudiera apagarse y encenderse *sin interrumpir el alunizaje* (Eyles, 2004). Para lograrlo, los programadores marcaron en el código *puntos de control* (*waypoints*), donde se registraba el avance de los trabajos esenciales.

Cuando aparecía una alarma, el software ejecutaba un reinicio controlado:

1. Vaciaba la lista de *jobs*, incluidas todas las copias atrasadas de SERVICER.
2. Volvía a iniciar solo los trabajos protegidos, cada uno desde su último punto de control. Si había varias copias del mismo trabajo, solo se reiniciaba la más reciente.
3. Los trabajos no esenciales, como una pantalla de monitoreo que había pedido la tripulación, simplemente desaparecían (Eyles, 2004).

Adler (1998), uno de los programadores de la AGC, lo compara con una computadora moderna que, después de reiniciarse, no vuelve a abrir el reproductor de música, pero sí regresa al mensaje que el usuario estaba escribiendo, con el mismo texto.

En la última fase del descenso, Neil Armstrong tomó el control semimanual de la nave, un modo que requería menos cálculos. A partir de ese momento ya no hubo alarmas, y el módulo lunar aterrizó a las 102 horas, 45 minutos y 40 segundos de misión (Mission Evaluation Team, 1969).

4.1.5. La decisión en Houston

Mientras tanto, en el centro de control, el oficial de guiado Steve Bales tenía segundos para decidir si se abortaba el alunizaje. Poco antes de la misión, su equipo había revisado qué significaba cada alarma y había concluido que la 1202 no era peligrosa mientras no fuera demasiado frecuente y la trayectoria de la nave se mantuviera correcta. Con el respaldo del ingeniero Jack Garman, Bales dio su aprobación (Eyles, 2004), y el comunicador Charlie Duke transmitió a la tripulación: «We're Go on that alarm» (E. M. Jones, 2018).

4.1.6. Mitos y precisiones

Este episodio suele contarse de forma simplificada. Conviene aclarar tres puntos:

- **No fue un error de programación.** Las alarmas eran la respuesta prevista del sistema ante una sobrecarga.

- **La causa exacta es discutida.** Eyles (2004) atribuye el problema al documento de interfaz que no exigía sincronizar las señales. En cambio, Margaret Hamilton, quien dirigió al equipo de software, lo atribuye a un documento de procedimientos de los astronautas que indicaba una posición incorrecta del interruptor del radar (MIT News, 2009). Ambas son fuentes de primera mano.
- **Fue un trabajo colectivo.** Laning diseñó el *Executive*, el equipo de Hamilton desarrolló el software de vuelo y su manejo de errores, y en Houston la decisión dependió de Bales y Garman. Curiosamente, Eyles señala que la protección de reinicio se había creado pensando en fallas de energía, no en este tipo de sobrecarga.

4.1.7. Relación con el curso

Concepto del curso	**Equivalente en la AGC**
Proceso	*Job*
Bloque de control de proceso	*Core set* y *VAC area*
Planificación por prioridades	El *Executive*
Interrupciones del temporizador	El *Waitlist*
Tabla de procesos llena	Alarmas 1201 y 1202
Inanición de un proceso	SERVICER, de baja prioridad, nunca terminaba
Recuperación con puntos de control	Reinicio desde los *waypoints*

Tabla 4: Conceptos del curso presentes en el caso del Apolo 11.

La lección del Apolo 11 responde directamente a la pregunta de este trabajo: el sistema no evitó la falla, pero estaba diseñado para *fallar bien*. En lugar de colapsar, sacrificó lo que no era indispensable y protegió lo que sí lo era.

4.2. Mars Pathfinder (1997): la inversión de prioridades llega a Marte

4.2.1. Contexto

El Mars Pathfinder aterrizó en Marte el 4 de julio de 1997 y llevaba consigo a Sojourner, el primer vehículo con ruedas en recorrer la superficie de otro planeta. Su computadora usaba el sistema operativo VxWorks, descrito en la sección anterior, que planifica los hilos por prioridades con apropiación (G. E. Reeves, 1997).

Pocos días después del aterrizaje, la nave comenzó a reiniciarse por sí sola, una y otra vez. Cada reinicio borraba las actividades programadas para ese día, aunque conservaba los datos ya recolectados. El problema resultó ser exactamente el que se describió en la sección 2: una inversión de prioridades.

La explicación más conocida de este caso es un resumen de Mike Jones, basado en una conferencia (M. Jones, 1997). Sin embargo, Glenn Reeves, líder del software de vuelo en el Jet Propulsion Laboratory (JPL), escribió después un relato de primera mano que corrige varios detalles (G. E. Reeves, 1997). Esta sección se basa principalmente en el relato de Reeves.

4.2.2. Las tareas involucradas

La computadora se comunicaba con los instrumentos de la nave mediante un bus de datos que trabajaba en ciclos de ocho por segundo, es decir, cada 0.125 segundos. Tres tareas de VxWorks participaron en el problema (G. E. Reeves, 1997):

Tarea	**Prioridad**	**Función**
`bc_sched`	Máxima	Prepara las transmisiones del bus y vigila el ciclo
`bc_dist`	Alta	Distribuye los datos recibidos por el bus
ASI/MET	Baja	Recolecta datos meteorológicos

Tabla 5: Tareas que intervinieron en la falla del Mars Pathfinder.

Además, había varias tareas de prioridad media que no tenían relación con el bus. Las tareas `bc_dist` y ASI/MET compartían un recurso protegido por un mutex. Este mutex no estaba en el código de la misión, sino dentro de un mecanismo de comunicación del propio VxWorks, la función `select()`, que ambas tareas usaban para intercambiar datos (G. E. Reeves, 1997).

4.2.3. Qué pasó

La secuencia fue la siguiente:

1. ASI/MET, de baja prioridad, tomó el mutex.
2. Antes de que lo liberara, se volvieron listas varias tareas de prioridad media. Como tenían más prioridad que ASI/MET, el planificador las ejecutó.
3. `bc_dist`, de alta prioridad, intentó tomar el mutex para enviar datos, pero ASI/MET lo tenía. `bc_dist` se bloqueó.

4. Las tareas medias siguieron ejecutándose, ASI/MET no avanzaba y, por lo tanto, nunca liberaba el mutex.

5. Al iniciar el siguiente ciclo, `bc_sched` revisó si `bc_dist` había terminado su trabajo. Como no era así, declaró un error grave y reinició la computadora (G. E. Reeves, 1997).

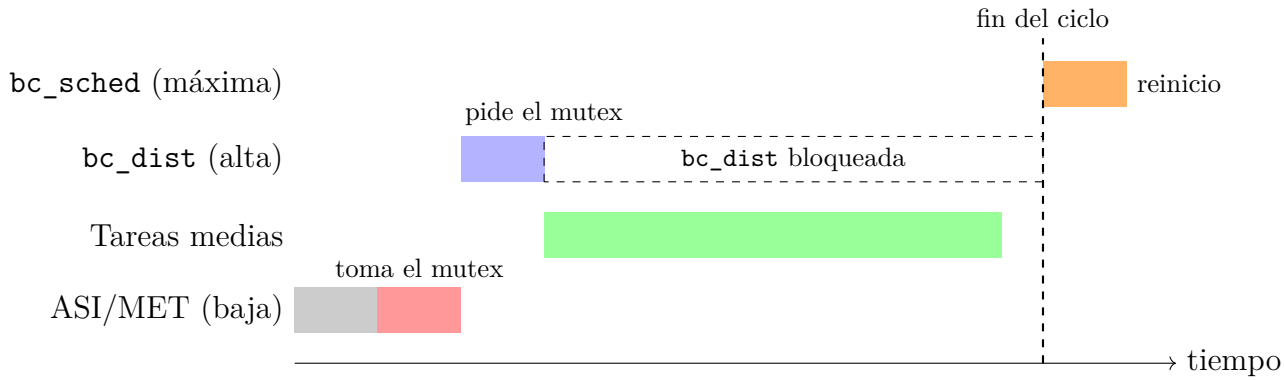


Figura 3: Inversión de prioridades en el Mars Pathfinder dentro de un ciclo del bus. En rojo, la sección crítica de ASI/MET. Al final del ciclo, `bc_sched` detecta que `bc_dist` no terminó y reinicia el sistema.

Es el mismo patrón de las tareas A, M y B de la sección 2, con un detalle adicional: aquí la tarea de alta prioridad tenía un plazo estricto, y el sistema estaba diseñado para reiniciarse si no lo cumplía.

4.2.4. El reinicio como protección

Muchos libros dicen que un *watchdog* reinició la nave. Según Reeves, el reinicio lo provocó en realidad `bc_sched`, que revisaba en cada ciclo que `bc_dist` hubiera terminado a tiempo (G. E. Reeves, 1997). En la práctica, `bc_sched` funcionaba como un *watchdog* implementado en software.

Este reinicio no fue el problema, sino parte de la solución. El sistema estaba diseñado para soportar reinicios múltiples, incluso durante la entrada a la atmósfera de Marte. Gracias a él, la nave nunca quedó en un estado descontrolado: cada vez que algo salía mal, regresaba a un estado conocido y seguro.

4.2.5. El diagnóstico

VxWorks incluía una herramienta que registraba en memoria todos los eventos importantes del sistema: cambios de contexto, uso de mutex, semáforos e interrupciones. Además, `bc_sched` estaba programada para guardar ese registro cuando detectara el error. Con esa información,

los ingenieros del JPL reprodujeron la falla en una réplica de la nave en la Tierra en menos de 18 horas (G. E. Reeves, 1997).

4.2.6. La corrección

VxWorks permite crear un mutex con o sin herencia de prioridad, y el mutex de `select()` tenía esta opción desactivada. Los ingenieros encontraron que Wind River había dejado la configuración de ese mutex en variables globales, por lo que podían modificarla sin cambiar el código de VxWorks (G. E. Reeves, 1997).

El cambio consistió en activar la herencia de prioridad. Como afectaba a todos los mutex de `select()`, no solo al problemático, el JPL lo analizó y lo probó cuidadosamente antes de aplicarlo. Después, se envió a la nave un archivo con las diferencias entre el software a bordo y el corregido, y un programa de la propia nave verificó el cambio y lo aplicó en memoria (G. E. Reeves, 1997). A partir de entonces, los reinicios desaparecieron.

4.2.7. Más allá de la versión de los libros

La versión popular del caso contiene dos imprecisiones que Reeves corrigió por escrito:

- **Cómo se aplicó el parche.** M. Jones (1997) contó que la corrección se cargó con el intérprete de comandos de VxWorks. Reeves aclaró que no fue así: se usó el mecanismo de actualización propio de la nave, que validaba los cambios antes de aplicarlos (G. E. Reeves, 1997).
- **Las herramientas de diagnóstico no estaban ahí por suerte.** Jones afirmó que las herramientas de depuración se habían dejado activas «afortunadamente». Reeves respondió que permanecían en el software por diseño (G. E. Reeves, 1997).

También es interesante saber por qué el problema no se detectó antes del lanzamiento. Solo aparecía cuando ASI/MET estaba activa y las tareas medias tenían mucha carga, y en la superficie de Marte la cantidad de datos fue mayor de lo previsto. El equipo sí había visto reinicios similares en pruebas, pero no logró reproducirlos y les dio menor prioridad que al software de aterrizaje, que era más urgente (G. E. Reeves, 1997). La lección que el propio Reeves dejó para quienes usan software comercial es sencilla: hay que entender cómo funciona.

4.2.8. El debate: ¿herencia o techo de prioridad?

La herencia de prioridad resolvió el problema del Pathfinder, pero no es una solución indiscutida.

- **A favor.** Fue el parche ideal para una nave ya en vuelo: era un cambio mínimo, se activaba con una opción de configuración y no requería reescribir el código (Sha et al., 1990).
- **En contra.** Yodaiken (2004) sostiene que la herencia de prioridad no es eficiente ni confiable. Cuando una tarea tiene varios mutex a la vez, la herencia debe aplicarse en cadena y es difícil hacerlo correctamente; si una tarea pierde la prioridad heredada antes de tiempo, la inversión reaparece. Su recomendación es diseñar secciones críticas cortas y fáciles de analizar.
- **Una prueba formal.** Zhang et al. (2020) verificaron el protocolo con un demostrador automático de teoremas y encontraron que la demostración original de 1990 tenía un error en el caso de varios mutex. El protocolo puede implementarse correctamente, pero no es tan sencillo como parece.

El *techo de prioridad*, propuesto en el mismo artículo de 1990, es más estricto y evita tanto la inversión no acotada como los bloqueos mutuos (Sha et al., 1990). La idea de fondo incluso es anterior: los diseñadores del lenguaje Mesa ya proponían asignar a cada recurso compartido la prioridad del proceso más importante que pudiera usarlo (Lampson & Redell, 1980).

Una conclusión razonable es que la herencia de prioridad fue la mejor solución *para reparar* la nave en vuelo, pero que, al *diseñar* un sistema nuevo, es preferible evitar que tareas de prioridades muy distintas compartan recursos, o usar el techo de prioridad.

4.2.9. Relación con el curso

Concepto del curso	**En el Mars Pathfinder**
Exclusión mutua	El mutex de `select()`
Estados de un hilo	`bc_dist` pasa de ejecución a bloqueado
Planificación con apropiación	Las tareas medias desplazan a ASI/MET
Inversión de prioridades no acotada	`bc_dist` espera indefinidamente
Plazo de tiempo real duro	El ciclo de 0.125 segundos del bus
Watchdog	`bc_sched` y el reinicio

Tabla 6: Conceptos del curso presentes en el caso del Mars Pathfinder.

El Pathfinder responde a las dos partes de la pregunta de este trabajo. Cuando no podía fallar, el sistema dependía de que la planificación por prioridades funcionara correctamente.

Cuando falló de todos modos, un mecanismo de vigilancia lo devolvió a un estado seguro, y las herramientas de diagnóstico, presentes por diseño, permitieron corregirlo a millones de kilómetros de distancia.

4.3. Radiación: lo que hace el software cuando el hardware no alcanza

4.3.1. Qué le corresponde al hardware y qué al software

Como se explicó en la sección 2, la radiación puede cambiar bits en la memoria de una nave. La primera defensa contra este problema es el hardware: procesadores fabricados para resistir la radiación, memorias con código de corrección de errores (ECC) y circuitos duplicados que comparan sus resultados. Estas protecciones no forman parte del sistema operativo y, por lo tanto, solo se mencionan aquí como contexto.

Sin embargo, el hardware no resuelve todo. Algunas misiones usan procesadores comerciales, más rápidos y baratos que los resistentes a la radiación, pero también más vulnerables. Incluso con ECC, los errores pueden acumularse si nadie revisa la memoria. En estos casos, el software debe completar lo que el hardware no alcanza a hacer. Esta sección presenta dos misiones que lo demuestran con datos reales.

4.3.2. CALIPSO: el barrido de memoria como tarea del sistema

CALIPSO es un satélite de la NASA y la agencia espacial francesa que estudia las nubes y los aerosoles de la atmósfera. Fue lanzado en abril de 2006 y su computadora usa procesadores comerciales PowerPC, no diseñados para el espacio (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008).

Su memoria tenía ECC, pero con una limitación importante: cuando el hardware detectaba y corregía un error durante una lectura, entregaba el valor correcto al procesador, pero *no lo reescribía en la memoria*. El dato dañado seguía ahí, y si después aparecía un segundo error en la misma palabra, ya no podría corregirse (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008).

Para resolverlo, el software de vuelo incluía una tarea llamada *Memory Error Scrubbing*, que funcionaba así (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008):

1. Dividía la memoria en uso en 128 bloques de 512 KB.

2. Leía cada bloque completo, aprovechando que el ECC revisa todo lo que se lee.

3. Si el ECC detectaba un error, se generaba una interrupción. La rutina que la atendía corregía el dato en memoria y registraba el evento.

4. Si un bloque tenía errores, se volvía a leer hasta 12 veces para asegurarse de que quedara limpio.

5. Toda la memoria se recorría cada 10 minutos.

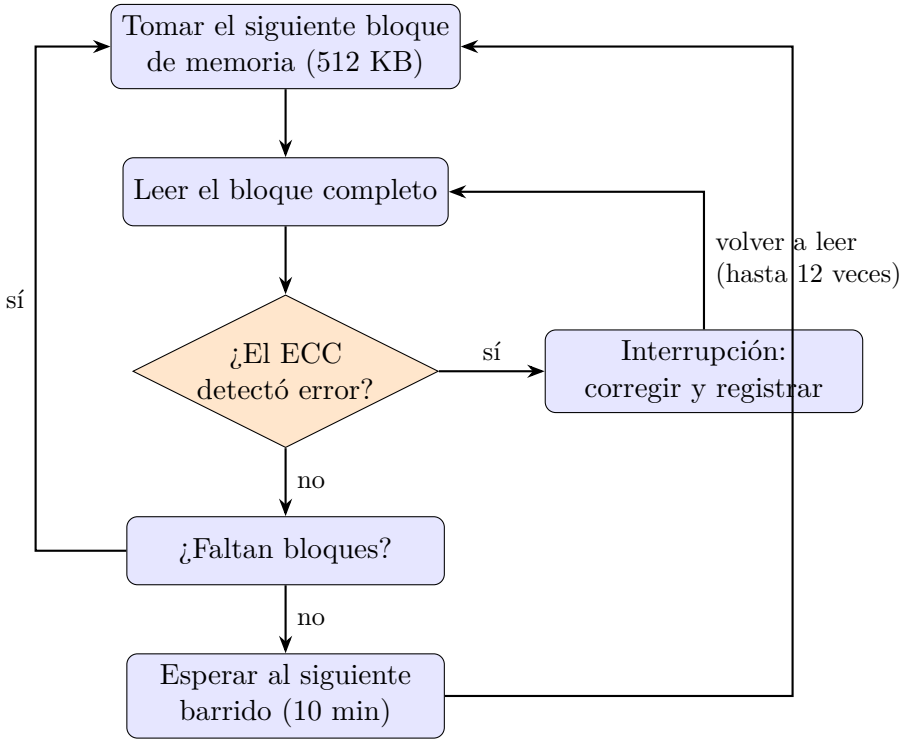


Figura 4: Funcionamiento de la tarea de barrido de memoria de CALIPSO, según *Non radiation hardened microprocessors in space based remote sensing systems* (2008).

Los resultados muestran que la técnica funcionó y que su costo fue mínimo. Entre el 4 de mayo y el 29 de julio de 2006, es decir, en 87 días, la tarea corrigió 493 errores causados por radiación, y solo usó el 0.004 % del tiempo del procesador (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008).

CALIPSO también muestra otros mecanismos de recuperación por software:

- **Resincronización de procesadores.** La computadora tenía cuatro procesadores que ejecutaban lo mismo y comparaban resultados. Si uno discrepaba, el hardware lo marcaba, y una vez por segundo el software revisaba esa marca y resincronizaba al procesador afectado en aproximadamente un milisegundo, sin detener al resto. En los 87 días hubo 9 discrepancias (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008).

- **Arranque de respaldo.** Si la radiación dañaba el código de arranque, el *watchdog* detectaba que el sistema no respondía, lo reiniciaba y la computadora arrancaba desde una copia de respaldo (*Non radiation hardened microprocessors in space based remote sensing systems*, 2008).

Desde el punto de vista del curso, la tarea de barrido es un ejemplo interesante de planificación: es una tarea periódica, de baja prioridad y que consume muy poco, pero que protege la memoria de todas las demás.

4.3.3. ARGOS: proteger el propio programa

ARGOS fue un satélite experimental de la Fuerza Aérea de Estados Unidos, lanzado en 1999, que llevaba procesadores comerciales para probar cómo se comportaban en el espacio. En uno de sus experimentos, la memoria no tenía protección de hardware, y los errores por radiación llegaban a dañar el sistema operativo o los programas, lo que obligaba a reiniciar con frecuencia (Shirvani et al., 2000).

Para resolverlo, Shirvani et al. (2000) implementaron el código de corrección de errores *en software*. El programa guardaba información adicional sobre el código del sistema y la revisaba periódicamente para detectar y reparar bits alterados. Los autores concluyen que es una solución de bajo costo que mejora la disponibilidad del sistema, sin necesidad de agregar hardware.

Otra técnica probada en ARGOS fue la duplicación de instrucciones (*Error Detection by Duplicated Instructions*, EDDI). Al compilar el programa, cada operación importante se duplica y, después, se comparan los dos resultados; si no coinciden, significa que la radiación alteró algo. En experimentos de inyección de fallas, esta técnica detectó más del 98 % de los errores sin hardware adicional (Oh et al., 2002).

4.3.4. Otras técnicas de software

Además del barrido de memoria y de la protección del código, las naves combinan otras técnicas que ya se mencionaron en este trabajo:

- **Votación por mayoría.** El software guarda tres copias de los datos importantes y, al leerlos, elige el valor que aparezca al menos dos veces (NASA, 2021).
- **Componentes reiniciables.** Si la radiación provoca que un componente se trabe, mecanismos como el componente *Health* de F' o los servicios de reinicio de cFS, descritos en la sección 3, lo detectan y lo reinician sin detener al resto del sistema.

4.3.5. Relación con el curso

Concepto del curso	En la mitigación de radiación
Administración de memoria	El barrido de memoria de CALIPSO
Tareas periódicas	El barrido se repite cada 10 minutos
Planificación por prioridades	El barrido corre con baja prioridad y consume muy poco
Interrupciones	El ECC avisa los errores mediante interrupciones
Watchdog y reinicio	El arranque de respaldo de CALIPSO
Aislamiento de componentes	*Health* de F' y reinicio de aplicaciones de cFS

Tabla 7: Conceptos del curso presentes en la mitigación de errores por radiación.

Este caso responde a la observación de que muchas defensas de una nave dependen del hardware. Es cierto, pero el hardware solo detecta o corrige errores en el momento en que ocurren. El software, y en particular las tareas del sistema, es lo que garantiza que esos errores no se acumulen, que se registren y que, si algo falla de todos modos, el sistema pueda volver a un estado seguro.

4.4. Spirit (2004): cuando reiniciar no basta

4.4.1. Contexto

El rover Spirit llegó a Marte en enero de 2004, como parte de la misión Mars Exploration Rover, junto con su gemelo Opportunity. Su computadora usaba VxWorks (Klemm & Reeves, 2005) y guardaba sus archivos en una memoria FLASH, un tipo de memoria que conserva los datos aunque se apague el equipo, como la de una memoria USB.

En las misiones marcianas el tiempo se mide en *soles*, que son días de Marte, ligeramente más largos que los de la Tierra. En el sol 18, el 21 de enero de 2004, Spirit dejó de comunicarse con normalidad y comenzó a reiniciarse una y otra vez. Permaneció fuera de control hasta el sol 21, y su actividad científica normal no se reanudó hasta el sol 33 (G. Reeves & Neilson, 2005).

4.4.2. La causa: un recurso que se agotaba

Según el análisis de G. Reeves y Neilson (2005), la causa principal fue un error de diseño en el módulo del sistema operativo que administraba el sistema de archivos, combinado con dos errores de configuración. Para entenderlo, conviene usar una analogía.

Imaginemos un cuaderno con un índice al principio. Cuando borramos un archivo, en lugar de arrancar la hoja, solo lo tachamos en el índice. El archivo ya no se usa, pero su

renglón en el índice sigue ocupando espacio. El sistema de archivos de Spirit funcionaba así: los archivos borrados solo se marcaban como eliminados, y al arrancar el sistema copiaba el índice completo a la memoria RAM, incluidos los renglones tachados. Lo que importaba no era cuántos archivos había en ese momento, sino cuántos había llegado a haber como máximo (G. Reeves & Neilson, 2005).

A esto se sumaron dos errores de configuración (G. Reeves & Neilson, 2005):

1. El espacio de RAM reservado para el sistema de archivos podía crecer sin ningún límite, tomando memoria del resto del sistema.

2. Cuando una tarea pedía memoria y ya no había disponible, el sistema no reportaba un error: simplemente suspendía la tarea *sin avisar*.

En el sol 18 se habían creado muchos archivos nuevos. El índice creció tanto que agotó la memoria, y la tarea que estaba trabajando con el sistema de archivos quedó suspendida.

4.4.3. El ciclo de reinicios

Spirit tenía un monitor de salud que detectaba tareas suspendidas y, al encontrar una, reiniciaba la computadora. En otro contexto esto habría resuelto el problema, pero aquí ocurrió lo contrario: al arrancar, el sistema volvía a montar la memoria FLASH, volvía a copiar el índice completo a la RAM, la memoria volvía a agotarse y el ciclo se repetía (G. Reeves & Neilson, 2005).

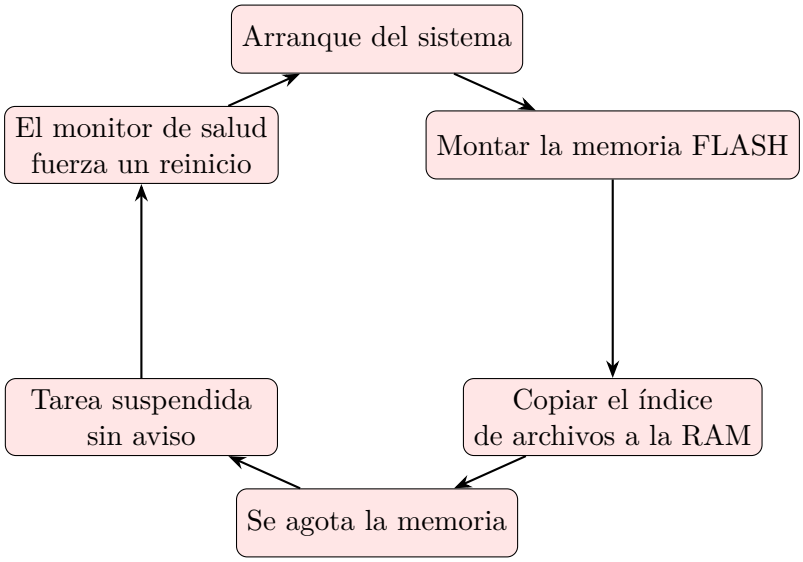


Figura 5: Ciclo de reinicios del rover Spirit. Cada reinicio volvía a provocar la falla que lo había causado.

El problema tuvo además un efecto secundario muy relacionado con el curso. La memoria se agotó justo cuando la tarea estaba dentro de una sección crítica protegida por un semáforo. Como la tarea quedó suspendida, nunca liberó el semáforo, y otras tareas que lo necesitaban se bloquearon, entre ellas la que apagaba el rover por las noches para ahorrar energía (G. Reeves & Neilson, 2005). Es un ejemplo de lo que ocurre cuando un proceso retiene un recurso y nunca lo libera: una situación muy cercana a un bloqueo mutuo.

4.4.4. La recuperación: el modo *crippled*

La solución vino de una idea heredada del Mars Pathfinder. Spirit tenía un modo de arranque llamado *crippled* («lisiado»), que se activaba con una orden procesada directamente por el hardware, sin pasar por el software dañado. Esa orden encendía un bit que el sistema revisaba al arrancar y que le indicaba no usar la memoria FLASH. En su lugar, el sistema creaba un sistema de archivos vacío y pequeño en la RAM, con el mismo nombre, de modo que el resto del software podía seguir funcionando sin notar la diferencia (G. Reeves & Neilson, 2005).

El sistema de archivos temporal era diez veces más pequeño que el original y se borraba al apagar el rover, por lo que había que enviar la orden cada sol. Aun así, permitió recuperar el control en el sol 21. Después, los ingenieros borraron datos que ya no se necesitaban, y en el sol 32 borraron y formatearon por completo la memoria FLASH, verificando en cada paso que el software de vuelo siguiera intacto (G. Reeves & Neilson, 2005). Los autores son claros sobre la importancia de este mecanismo: sin él, la misión podría haberse perdido.

4.4.5. Lecciones

Después de la falla, el equipo modificó el software y sus procedimientos (G. Reeves & Neilson, 2005):

- El sistema compacta las entradas de archivos borrados antes de montar la memoria.
- El modo *crippled* se activa automáticamente después de varios reinicios seguidos.
- El semáforo que bloqueó el apagado nocturno ahora tiene un tiempo máximo de espera.
- Se detiene toda actividad si la memoria libre baja de cierto nivel.

Los autores también dejan lecciones generales: entender a fondo el software comercial que se usa, no permitir que el sistema reserve memoria después de arrancar y enviar a la Tierra información sobre el uso de los recursos. De hecho, la fuga de memoria ya podía verse en los datos de pruebas anteriores, pero nadie la había detectado (G. Reeves & Neilson, 2005).

4.4.6. Relación con el curso

Concepto del curso	En el rover Spirit
Administración de memoria	El índice de archivos agotó la RAM
Estados de un proceso	La tarea pasó a suspendida sin aviso
Semáforos y secciones críticas	El semáforo nunca liberado
Bloqueo de procesos	Las tareas que esperaban ese semáforo
Watchdog y reinicio	El monitor de salud y el ciclo de reinicios
Recuperación en modo degradado	El modo *crippled*

Tabla 8: Conceptos del curso presentes en el caso del rover Spirit.

El contraste con el Apolo 11 es instructivo. En la computadora del Apolo, reiniciar *liberaba* los recursos agotados y permitía continuar. En Spirit, reiniciar los *volvía a consumir*. Un reinicio solo es una buena estrategia de recuperación si elimina la causa del problema. Cuando no lo hace, el sistema necesita una alternativa, como el modo *crippled*, que le permita arrancar sin el componente dañado.

4.5. Ingenuity (2021): Linux en Marte

4.5.1. Contexto

Ingenuity fue un pequeño helicóptero que viajó a Marte con el rover Perseverance. El 19 de abril de 2021 se convirtió en la primera aeronave en realizar un vuelo controlado en otro planeta. Era una *demostración tecnológica*: su objetivo era probar que se podía volar en la delgada atmósfera marciana, por lo que la NASA aceptó correr más riesgos que en una misión principal (Tzanetos et al., 2022).

Lo que lo hace especial para este trabajo es su computadora. A diferencia de todos los casos anteriores, Ingenuity usó Linux, un sistema operativo de propósito general, sobre un procesador comercial de teléfono celular. Como se vio en la sección 3, la propia NASA reconoce que Linux no es un sistema de tiempo real (McComas, 2012). Entonces, ¿cómo pudo volar con él?

4.5.2. Una arquitectura dividida por niveles de criticidad

La respuesta es que Linux *no controlaba el vuelo*. La computadora de Ingenuity estaba dividida en tres niveles, cada uno con una responsabilidad distinta (Balaram et al., 2018; T. Canham, 2021):

- **Computadora de navegación.** Un procesador Qualcomm Snapdragon 801, del tipo que se usaba en teléfonos celulares, con Linux. Se encargaba de procesar las imágenes de la cámara para calcular el movimiento del helicóptero, además de la telemetría y las comunicaciones.
- **Computadora de vuelo.** Dos microcontroladores Texas Instruments Hercules, que ejecutaban el control de vuelo directamente sobre el hardware, sin sistema operativo. Ajustaban los motores 500 veces por segundo (Grip, 2021).
- **FPGA.** Un circuito resistente a la radiación que conectaba los sensores y los motores con las computadoras y supervisaba todo el sistema.

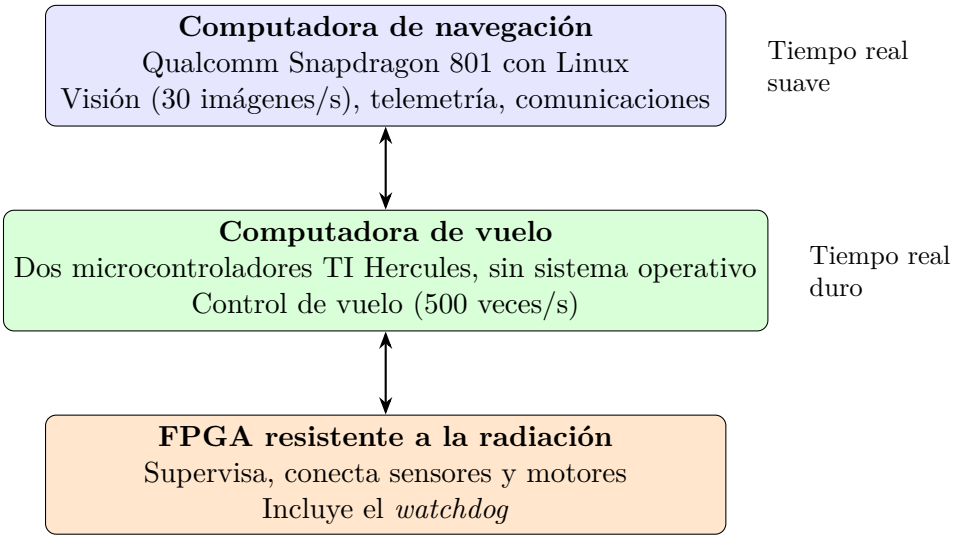


Figura 6: Arquitectura de cómputo de Ingenuity. Linux se usó solo en el nivel donde un retraso no es catastrófico.

Esta división aplica directamente los conceptos de la sección 2. El control de vuelo es tiempo real duro: si un ajuste llega tarde, el helicóptero puede perder la estabilidad. Por eso se dejó en procesadores sin sistema operativo, cuyo comportamiento es completamente predecible. El procesamiento de imágenes es tiempo real suave: la cámara envía 30 imágenes por segundo (Grip, 2021), y si una se procesa con un poco de retraso, la navegación empeora, pero el helicóptero no se cae. Para ese trabajo, que además necesita mucha potencia de cálculo, Linux era suficiente.

4.5.3. Cómo se configuró Linux

Timothy Canham, arquitecto del software de Ingenuity en el JPL, describió cómo adaptaron Linux a la misión (T. Canham, 2021):

- Usaron un kernel de Linux 3.4 con opciones para mejorar la respuesta, pero *sin* el parche de tiempo real.
- Cada uno de los cuatro núcleos del procesador se asignó a un tipo de trabajo, con la herramienta `taskset`: uno para datos y comunicaciones, otro para las cámaras, otro para el procesamiento de imágenes y otro para la navegación. Así, las tareas no competían por el mismo núcleo.
- El software de vuelo se basó en F', el *framework* del JPL descrito en la sección 3 (T. K. Canham, 2022).
- El programa se guardó en seis copias, cada una con una suma de verificación, para tener respaldos en caso de que la radiación dañara alguna.

El propio Canham reconoce los límites de esta decisión: como Linux no es de tiempo real, el software se diseñó para tolerar retrasos, y probablemente habría sido mejor usar el parche de tiempo real (T. Canham, 2021).

4.5.4. Incidente 1: el *watchdog* antes del primer vuelo

El 9 de abril de 2021, durante una prueba previa al primer vuelo, los rotores debían girar a alta velocidad. La prueba se detuvo antes de tiempo porque expiró un *watchdog* durante el cambio del modo de prevuelo al modo de vuelo (NASA/JPL, 2021a).

Meses después, el equipo explicó la causa: el *watchdog* estaba en la FPGA y vigilaba que la comunicación entre los microcontroladores de vuelo y la FPGA cumpliera tiempos muy estrictos. Durante el arranque, a veces esos tiempos no se cumplían, y el *watchdog* detenía todo por seguridad (Tzanetos, 2021). Es decir, no fue una falla de Linux, sino un problema de *temporización* en el arranque.

El equipo tenía dos opciones: reinstalar el software de los controladores de vuelo o cambiar la secuencia de comandos de arranque. Eligieron la segunda, que funcionaba aproximadamente el 85 % de las veces y, si fallaba, dejaba al helicóptero en un estado seguro. Prefirieron no reinstalar un software que llevaba casi dos años funcionando de forma estable (Aung, 2021). Con esa solución, Ingenuity voló por primera vez el 19 de abril. Más tarde, una actualización del software de los controladores corrigió el problema de forma permanente, lo que se confirmó en el vuelo 8 (Tzanetos, 2021).

4.5.5. Incidente 2: el vuelo 6 y las marcas de tiempo

El 22 de mayo de 2021, durante el sexto vuelo, ocurrió una falla más grave. A los 54 segundos de vuelo se perdió *una sola imagen* de la cámara de navegación. El problema fue

que, a partir de ese momento, todas las imágenes siguientes llegaron con una marca de tiempo equivocada (Grip, 2021).

El sistema de navegación compara cada imagen con la anterior para calcular el movimiento, y para ello necesita saber exactamente cuándo se tomó cada una. Con las marcas de tiempo desfasadas, el sistema creía ver errores que no existían y trataba de corregirlos, lo que hizo que el helicóptero oscilara con inclinaciones de más de 20 grados. Aun así, Ingenuity aterrizó a unos 5 metros del punto planeado (Grip, 2021).

El helicóptero se salvó por dos decisiones de diseño. El control de vuelo tenía un amplio margen de estabilidad y estaba diseñado para tolerar errores importantes, incluidos errores de tiempo. Además, en la fase final del descenso el sistema dejaba de usar las imágenes, precisamente para evitar este tipo de problemas (Grip, 2021).

Desde el punto de vista del curso, es un problema de sincronización entre un productor (la cámara) y un consumidor (la navegación). Un dato correcto con una marca de tiempo equivocada es, en la práctica, un dato incorrecto. La NASA no atribuyó la falla a Linux, por lo que no puede afirmarse que el sistema operativo la haya causado.

4.5.6. Relación con el curso

Concepto del curso	**En Ingenuity**
Tiempo real duro y suave	Control de vuelo frente a procesamiento de imágenes
Asignación de procesos a procesadores	`taskset` asigna un núcleo a cada tipo de tarea
Estructura del sistema	División en tres niveles de criticidad
Watchdog	La FPGA detuvo el arranque por tiempos incumplidos
Productor y consumidor	La cámara y la navegación en el vuelo 6

Tabla 9: Conceptos del curso presentes en el caso de Ingenuity.

Ingenuity muestra que la pregunta no es si conviene usar Linux o un sistema de tiempo real, sino *dónde* usar cada uno. Un sistema operativo de propósito general puede volar en Marte, siempre que se le asignen solo las tareas que toleran retrasos y que lo crítico quede en manos de componentes cuyo comportamiento sea completamente predecible.

5. Discusión

Los cinco casos analizados abarcan más de cincuenta años de exploración espacial, con computadoras, sistemas operativos y misiones muy distintas. Sin embargo, al compararlos aparecen patrones que se repiten. Esta sección los identifica y responde, con base en ellos, a la pregunta que guía este trabajo.

5.1. Resumen de los casos

Caso	**Sistema**	**Falla**	**Respuesta del sistema**	**Concepto**
Apolo 11 (1969)	*Executive* de la AGC	Sobrecarga por un radar mal sincronizado	Reinicio que descartó lo no esencial	Planificación por prioridades
Pathfinder (1997)	VxWorks	Inversión de prioridades	Reinicio por plazo incumplido y parche remoto	Exclusión mutua
CALIPSO (2006)	Software de vuelo	Errores por radiación	Barrido periódico de memoria	Administración de memoria
Spirit (2004)	VxWorks	Memoria agotada por el sistema de archivos	Ciclo de reinicios, resuelto con el modo *crippled*	Recursos y semáforos
Ingenuity (2021)	Linux y microcontroladores	Tiempos incumplidos y marcas de tiempo erróneas	*Watchdog* de la FPGA y margen de estabilidad	Tiempo real duro y suave

Tabla 10: Comparación de los casos de estudio.

5.2. Patrones comunes

5.2.1. Un vigilante y un reinicio controlado

En todos los casos aparece algún mecanismo que vigila al sistema y, si algo sale mal, lo devuelve a un estado conocido. En el Apolo 11 fue el software de reinicio con puntos de control; en el Pathfinder, la tarea `bc_sched`; en CALIPSO, el *watchdog* del arranque; en Spirit, el monitor de salud; y en Ingenuity, el *watchdog* de la FPGA. La idea de fondo es la misma: ante la duda, es preferible detenerse o reiniciar que seguir operando en un estado desconocido.

5.2.2. Reiniciar solo sirve si elimina la causa

El reinicio, sin embargo, no es una solución universal. La comparación entre el Apolo 11 y Spirit lo muestra con claridad. En el Apolo, reiniciar liberaba los recursos agotados y la

computadora podía continuar. En Spirit, cada reinicio volvía a cargar el índice de archivos y agotaba de nuevo la memoria. Por eso los sistemas necesitan, además del reinicio, un modo alternativo de arranque que evite el componente dañado, como el modo *crippled*, que precisamente surgió de la experiencia del Pathfinder (G. Reeves & Neilson, 2005).

5.2.3. Separar lo crítico de lo que no lo es

Varios casos muestran que una buena forma de proteger lo esencial es separarlo del resto. En el Apolo 11, solo los trabajos marcados como protegidos sobrevivían a un reinicio. En CALIPSO, el barrido de memoria corría con baja prioridad para no interferir con las tareas principales. En Ingenuity, la separación llegó al nivel del hardware: lo crítico se ejecutaba en procesadores sin sistema operativo, y lo que toleraba retrasos, en Linux. Los *frameworks* como cFS y F' aplican la misma idea al dividir el software en componentes aislados que pueden reiniciarse por separado.

5.2.4. Diagnóstico por diseño

En el Pathfinder, la falla pudo corregirse porque el sistema registraba sus propios eventos y guardaba ese registro al detectar el error. G. E. Reeves (1997) insiste en que esas herramientas estaban ahí por diseño, no por suerte. En Spirit, en cambio, la fuga de memoria ya aparecía en los datos de pruebas anteriores, pero nadie la había vigilado (G. Reeves & Neilson, 2005). En ambos casos la lección es la misma: un sistema que no puede repararse físicamente debe estar preparado para explicar qué le pasó.

5.2.5. Entender el software que se usa

Tanto en el Pathfinder como en Spirit, el problema no estaba en el código escrito por la misión, sino en componentes del sistema operativo comercial: el mutex dentro de la función `select()` y la biblioteca del sistema de archivos. Ambas fallas dependían de opciones de configuración que el equipo no había revisado a fondo. Usar software comercial ahorra tiempo, pero no elimina la responsabilidad de entender cómo funciona.

5.3. ¿Sistema operativo de propósito general o de tiempo real?

Ingenuity podría interpretarse como una señal de que los sistemas operativos de propósito general, como Linux, ya pueden sustituir a los de tiempo real en el espacio. Los casos analizados sugieren una conclusión más matizada.

La decisión no es elegir uno u otro para toda la nave, sino asignar a cada uno el tipo de trabajo que mejor puede hacer. Ingenuity voló con Linux porque los ingenieros se aseguraron

Aspecto	Propósito general (Linux)	Tiempo real (VxWorks, RTEMS)
Objetivo	Buen rendimiento en promedio	Respuesta dentro de un plazo garantizado
Hardware disponible	Amplio, incluido el comercial	Limitado, a menudo resistente a radiación
Protección de memoria	Separa usuario y núcleo	Con frecuencia un solo espacio sin protección
Uso adecuado	Tareas que toleran retrasos	Tareas críticas con plazos estrictos

Tabla 11: Comparación entre sistemas operativos de propósito general y de tiempo real en el contexto espacial.

de que Linux nunca estuviera a cargo de algo que no pudiera esperar. También vale la pena notar una ventaja que a veces se pasa por alto: Linux separa el espacio del usuario del núcleo, algo que muchos sistemas de tiempo real espaciales sacrifican para ganar velocidad y sencillez.

5.4. Qué le corresponde al hardware y qué al sistema operativo

Muchas de las defensas de una nave dependen del hardware, y conviene distinguirlas con claridad de las que corresponden al sistema operativo.

Defensa	Responsable	Ejemplo
Resistir la radiación	Hardware	Procesadores RAD750 y FPGA de Ingenuity
Detectar y corregir un bit al leerlo	Hardware (ECC)	Memoria de CALIPSO
Evitar que los errores se acumulen	Software del sistema	Barrido de memoria de CALIPSO
Decidir qué se ejecuta y cuándo	Sistema operativo	*Executive* de la AGC, VxWorks
Coordinar el acceso a recursos	Sistema operativo	Mutex con herencia de prioridad
Detectar que algo dejó de responder	Hardware y software	*Watchdog*, monitores de salud
Recuperarse de una falla	Sistema operativo y *framework*	Reinicios del cFE, modo *crippled*

Tabla 12: Distribución de responsabilidades entre el hardware y el software.

La tabla muestra que el hardware se ocupa sobre todo de *resistir* y *detectar* los problemas en el momento en que ocurren, mientras que el sistema operativo y el software de vuelo se

ocupan de *organizar*, *coordinar* y *recuperar*. Ninguna de las dos partes basta por sí sola: el ECC de CALIPSO necesitaba una tarea de software para que los errores no se acumularan, y el *watchdog* de Ingenuity, implementado en hardware, dependía de que el software cumpliera los tiempos que vigilaba.

5.5. Limitaciones de este trabajo

Conviene señalar algunas limitaciones de las fuentes utilizadas:

- Algunos datos sobre las misiones que usan cada sistema operativo provienen de las propias empresas que los venden, por lo que se usaron solo cuando una fuente independiente los confirmaba.
- En varios casos, las fuentes de primera mano no coinciden en detalles, como el número de *core sets* de la AGC o la causa exacta de las alarmas del Apolo 11. Estas diferencias se señalaron en el texto.
- La información sobre la configuración de Linux en Ingenuity proviene principalmente de una presentación de su arquitecto, que no es un artículo arbitrado, aunque se complementó con publicaciones formales de la NASA y del IEEE.

6. Conclusiones

Este trabajo partió de una pregunta: *¿qué hace el sistema operativo de una nave espacial cuando no puede fallar, y qué hace cuando falla de todos modos?* Los casos analizados permiten responder ambas partes.

Cuando no puede fallar, el sistema operativo de una nave se apoya en la previsibilidad. Usa planificación por prioridades con apropiación para que las tareas críticas siempre se atiendan primero, coordina el acceso a los recursos compartidos con mecanismos como los mutex con herencia de prioridad y realiza tareas de mantenimiento, como el barrido de memoria, que evitan que los errores se acumulen. Además, separa lo crítico de lo que no lo es, ya sea mediante prioridades, mediante componentes aislados o, como en Ingenuity, asignando cada tipo de trabajo a un procesador distinto.

Cuando falla de todos modos, el sistema operativo no intenta seguir como si nada hubiera pasado. Un mecanismo de vigilancia detecta el problema y lo devuelve a un estado conocido y seguro, conservando lo esencial y descartando lo que no lo es. Si el reinicio no basta, como ocurrió en Spirit, el sistema necesita un modo alternativo de arranque que evite el

componente dañado. Y en todos los casos, las herramientas de diagnóstico incluidas desde el diseño son las que permiten entender y corregir la falla a millones de kilómetros de distancia.

En pocas palabras, un buen sistema operativo espacial no es el que nunca falla, sino el que está diseñado para *fallar bien*.

Los casos también muestran que la seguridad de una nave no depende solo del sistema operativo. El hardware resiste la radiación y detecta los errores en el momento en que ocurren, mientras que el sistema operativo y el software de vuelo organizan el trabajo, coordinan los recursos y recuperan al sistema cuando algo sale mal. Ninguna de las dos partes basta por sí sola.

Por último, este trabajo muestra que los conceptos que se estudian en un curso de Sistemas Operativos no son solo teóricos. La exclusión mutua, la planificación por prioridades, los estados de los procesos, la administración de memoria y los semáforos aparecen en situaciones reales donde un error puede costar una misión completa. El caso del Mars Pathfinder lo resume bien: un problema que puede explicarse con tres tareas y un mutex en un pizarrón estuvo a punto de comprometer una misión en Marte, y se resolvió gracias a una idea, la herencia de prioridad, que había sido formalizada años antes (Sha et al., 1990).

Glosario

Apropiación (*preemption*) Capacidad del sistema operativo de interrumpir una tarea en ejecución para darle el procesador a otra de mayor prioridad.

Bit flip Cambio no deseado de un bit de 0 a 1 o de 1 a 0, causado, por ejemplo, por una partícula de radiación.

Bus de datos Canal de comunicación que conecta la computadora de una nave con sus instrumentos y otros equipos.

Core set Bloque de memoria de la computadora del Apolo donde se guardaban la prioridad y el estado de cada trabajo. Equivale al bloque de control de proceso.

ECC (*Error Correcting Code*) Código que agrega bits adicionales a los datos en memoria para detectar y corregir errores al leerlos.

Executive Parte del sistema operativo de la computadora del Apolo que administraba los trabajos según su prioridad.

FLASH Tipo de memoria que conserva los datos aunque se apague el equipo, como la de una memoria USB.

FPGA Circuito electrónico que puede configurarse para realizar funciones específicas de hardware. En Ingenuity supervisaba todo el sistema.

Framework Conjunto de herramientas y estructuras de software que organiza los programas de una aplicación. cFS y F' son *frameworks* de software de vuelo.

Herencia de prioridad (*priority inheritance*) Solución a la inversión de prioridades: la tarea que tiene un mutex recibe temporalmente la prioridad de la tarea más importante que lo espera.

Inversión de prioridades (*priority inversion*) Situación en la que una tarea de alta prioridad queda esperando a una de menor prioridad, porque esta tiene un recurso que la primera necesita.

Job Trabajo de cualquier duración administrado por el *Executive* de la computadora del Apolo. Equivale a un proceso.

***Memory scrubbing* (barrido de memoria)** Tarea que recorre periódicamente toda la memoria para corregir los errores antes de que se acumulen.

Microkernel Diseño de sistema operativo en el que el núcleo es muy pequeño y la mayoría de los servicios funcionan como componentes separados que se comunican por mensajes.

Modo *crippled* Modo de arranque del rover Spirit que evitaba usar la memoria FLASH dañada y la sustituía por un sistema de archivos temporal en RAM.

Mutex Mecanismo que garantiza la exclusión mutua: solo una tarea a la vez puede tenerlo y entrar a la sección crítica que protege.

OSAL (*Operating System Abstraction Layer*) Capa de cFS que traduce una misma petición a la llamada correspondiente de cada sistema operativo.

Planificador (*scheduler*) Parte del sistema operativo que decide qué tarea usa el procesador en cada momento.

Plazo (*deadline*) Momento límite en el que una tarea de tiempo real debe terminar su trabajo.

Productor y consumidor Problema de sincronización entre una tarea que genera datos y otra que los usa. En Ingenuity, la cámara y la navegación.

RTOS (*Real-Time Operating System*) Sistema operativo de tiempo real, diseñado para garantizar que las tareas respondan dentro de sus plazos.

Sección crítica Parte de un programa que accede a un recurso compartido y que solo una tarea puede ejecutar a la vez.

Semáforo Mecanismo de sincronización que controla el acceso a recursos compartidos mediante un contador.

SEU (*Single Event Upset*) Error causado cuando una partícula de radiación altera un bit en un circuito electrónico.

Software Bus Servicio de cFS que permite a las aplicaciones comunicarse publicando mensajes y suscribiéndose a ellos.

Sol Día marciano, ligeramente más largo que un día terrestre.

Task En la computadora del Apolo, rutina corta programada para ejecutarse en un momento exacto. La administraba el *Waitlist*.

Techo de prioridad (*priority ceiling*) Solución a la inversión de prioridades: cada mutex recibe de antemano la prioridad de la tarea más importante que pueda usarlo.

Tiempo real duro (*hard real-time*) Sistema en el que incumplir un plazo, aunque sea una vez, puede provocar una falla grave.

Tiempo real suave (*soft real-time*) Sistema en el que incumplir un plazo de vez en cuando empeora el resultado, pero no lo arruina.

Waitlist Parte del sistema operativo de la computadora del Apolo que ejecutaba tareas cortas en momentos exactos, mediante interrupciones del temporizador.

Watchdog Temporizador que el software debe reiniciar periódicamente. Si no lo hace, el sistema concluye que algo falló y reinicia el equipo o el componente afectado.

Referencias

Adler, P. (1998). *Apollo 11 program alarms* (E. M. Jones, Ed.). Consultado el 11 de octubre de 2026, desde https://www.nasa.gov/wp-content/uploads/static/history/alsj/a11/a11.1201-pa.html

Aung, M. (2021, 17 de abril). *Why we choose to try our first helicopter flight on Monday.* Consultado el 11 de octubre de 2026, desde https://science.nasa.gov/blog/why-we-choose-to-try-our-first-helicopter-flight-on-monday/

Balaram, J., Canham, T., Duncan, C., Golombek, M., Grip, H. F., Johnson, W., Maki, J., Quon, A., Stern, R., & Zhu, D. (2018). Mars helicopter technology demonstrator. *2018 AIAA Atmospheric Flight Mechanics Conference*. https://doi.org/10.2514/6.2018-0023

Berger, R. W., et al. (2001). The RAD750: A radiation hardened PowerPC processor for high performance spaceborne applications. *2001 IEEE Aerospace Conference Proceedings*, *5*, 2263-2272. https://doi.org/10.1109/AERO.2001.931184

Bocchino, R. L., Jr., Canham, T. K., Watney, G. J., Reder, L. J., & Levison, J. W. (2018). F Prime: An open-source framework for small-scale flight software systems. *32nd Annual AIAA/USU Conference on Small Satellites*, (SSC18-XII-04). Consultado el 10 de octubre de 2026, desde https://digitalcommons.usu.edu/smallsat/2018/all2018/328/

Burkey, R. (s.f.). *Luminary 99: EXECUTIVE.agc*. Virtual AGC Project. Consultado el 11 de octubre de 2026, desde https://www.ibiblio.org/apollo/listings/LUM99R2/EXECUTIVE.agc.html

Canham, T. (2021). Linux on Mars. Consultado el 11 de octubre de 2026, desde https://static.sched.com/hosted_files/osselc21/b3/1.%20Timothy%20Canham.pdf

Canham, T. K. (2022). The Mars Ingenuity helicopter: A victory for open-source software. *2022 IEEE Aerospace Conference (AERO)*. https://doi.org/10.1109/AERO53065.2022.9843438

Core Flight System. (s.f.). *Core Flight System*. Consultado el 10 de octubre de 2026, desde https://www.coreflightsystem.net/

European Space Agency. (2026). *RTEMS SMP qualification data pack*. Consultado el 10 de octubre de 2026, desde https://rtems-qual.io.esa.int/

Eyles, D. (2004). Tales from the lunar module guidance computer. *27th Annual AAS Guidance and Control Conference*, (AAS 04-064). Consultado el 11 de octubre de 2026, desde https://klabs.org/history/apollo_11_alarms/eyles_2004/eyles_2004.htm

Grip, H. F. (2021, 27 de mayo). *Surviving an in-flight anomaly: What happened on Ingenuity's sixth flight*. Consultado el 11 de octubre de 2026, desde https://science.nasa.gov/blog/surviving-an-in-flight-anomaly-what-happened-on-ingenuitys-sixth-flight/

Jet Propulsion Laboratory. (2021). *Mars 2020 Perseverance landing press kit: Perseverance rover*. Consultado el 10 de octubre de 2026, desde https://www.jpl.nasa.gov/news/press_kits/mars_2020/landing/mission/spacecraft/perseverance_rover/

Jones, E. M. (2018). *The first lunar landing*. Consultado el 11 de octubre de 2026, desde https://apollojournals.org/alsj/a11/a11.landing.html

Jones, M. (1997). What really happened on Mars Rover Pathfinder. *The RISKS Digest*, *19*(49). Consultado el 11 de octubre de 2026, desde http://eos.cs.ovgu.de/eos_old/lehre/WS0708/vl_pkes/folien/risk.pathfinder.pdf

Klemm, R., & Reeves, G. (2005). VxWorks on the Mars exploration rovers. *Wind River Worldwide User Conference*. Consultado el 11 de octubre de 2026, desde https://ntrs.nasa.gov/citations/20060042824

Lampson, B. W., & Redell, D. D. (1980). Experience with processes and monitors in Mesa. *Communications of the ACM*, *23*(2), 105-117. https://doi.org/10.1145/358818.358824

McComas, D. (2012). NASA/GSFC's flight software Core Flight System. Consultado el 10 de octubre de 2026, desde https://ntrs.nasa.gov/citations/20130013412

Mindell, D. A. (2008). *Digital Apollo: Human and machine in spaceflight*. MIT Press.

Mission Evaluation Team. (1969, noviembre). *Apollo 11 mission report* (Technical Memorandum N.º MSC-00171). NASA Manned Spacecraft Center. Consultado el 11 de octubre de 2026, desde https://ntrs.nasa.gov/citations/19700008096

MIT News. (2009). Recalling the "giant leap". *MIT News*. Consultado el 11 de octubre de 2026, desde https://news.mit.edu/2009/apollo-vign-0717

NASA. (2021). Small spacecraft avionics. En *State-of-the-art of small spacecraft technology*. Consultado el 11 de octubre de 2026, desde https://www.nasa.gov/wp-content/uploads/2021/10/8.soa_avionics_2021.pdf

NASA Jet Propulsion Laboratory. (s.f.). *Svc::Health component*. Consultado el 10 de octubre de 2026, desde https://fprime.jpl.nasa.gov/latest/Svc/Health/docs/sdd/

NASA/JPL. (2021a, 12 de abril). *Work progresses toward Ingenuity's first flight on Mars*. Consultado el 11 de octubre de 2026, desde https://science.nasa.gov/blog/work-progresses-toward-ingenuitys-first-flight-on-mars/

NASA/JPL. (2021b, 8 de julio). *Meet the open-source software powering NASA's Ingenuity Mars helicopter*. Consultado el 10 de octubre de 2026, desde https://www.jpl.nasa.gov/news/meet-the-open-source-software-powering-nasas-ingenuity-mars-helicopter/

Non radiation hardened microprocessors in space based remote sensing systems (Reporte técnico N.º NTRS 20080014189). (2008). NASA. Consultado el 11 de octubre de 2026, desde https://ntrs.nasa.gov/api/citations/20080014189/downloads/20080014189.pdf

OAR Corporation. (2000). *What does RTEMS stand for?* Consultado el 10 de octubre de 2026, desde https://docs.rtems.org/releases/4.5.1-pre3/rtemsdoc/html/FAQ/FAQ00002.html

Oh, N., Shirvani, P. P., & McCluskey, E. J. (2002). Error detection by duplicated instructions in super-scalar processors. *IEEE Transactions on Reliability*, *51*(1), 63-75. https://doi.org/10.1109/24.994913

Rabideau, G., Benowitz, E., & Chien, S. (2017). Prototyping an onboard scheduler for the Mars 2020 rover. *International Workshop on Planning and Scheduling for Space (IWPSS)*.

Consultado el 10 de octubre de 2026, desde https://ai.jpl.nasa.gov/public/documents/papers/rabideau-iwpss2017-prototyping.pdf

Reeves, G., & Neilson, T. (2005). The Mars rover Spirit FLASH anomaly. *2005 IEEE Aerospace Conference*, 4186-4199. https://doi.org/10.1109/AERO.2005.1559723

Reeves, G. E. (1997, 15 de diciembre). *What really happened on Mars? Authoritative account*. Consultado el 10 de octubre de 2026, desde https://www.cs.unc.edu/~anderson/teach/comp790/papers/mars_pathfinder_long_version.html

Sha, L., Rajkumar, R., & Lehoczky, J. P. (1990). Priority inheritance protocols: An approach to real-time synchronization. *IEEE Transactions on Computers*, *39*(9), 1175-1185. https://doi.org/10.1109/12.57058

Sherrill, J. (2016). FreeBSD and RTEMS, Unix in a real-time operating system. *FreeBSD Journal*. Consultado el 10 de octubre de 2026, desde https://freebsdfoundation.org/wp-content/uploads/2016/08/FreeBSD-and-RTEMS-Unix-in-a-Real-Time-Operating-System.pdf

Shirvani, P. P., Saxena, N. R., & McCluskey, E. J. (2000). Software-implemented EDAC protection against SEUs. *IEEE Transactions on Reliability*, *49*(3), 273-284. https://doi.org/10.1109/24.914544

Tzanetos, T. (2021, 25 de junio). *Flight 8 success, software updates, and next steps*. Consultado el 11 de octubre de 2026, desde https://science.nasa.gov/blog/flight-8-success-software-updates-and-next-steps/

Tzanetos, T., Aung, M., Balaram, J., Grip, H. F., Karras, J. T., Canham, T. K., et al. (2022). Ingenuity Mars helicopter: From technology demonstration to extraterrestrial scout. *2022 IEEE Aerospace Conference (AERO)*. https://doi.org/10.1109/AERO53065.2022.9843428

Wind River Systems. (2002). *VxWorks programmer's guide 5.5*. Consultado el 10 de octubre de 2026, desde https://prex.jlab.org/wiki/images/8/86/Vxworks_programmers_guide5.5.pdf

Yodaiken, V. (2004). *Against priority inheritance* (Reporte técnico). Finite State Machine Labs. Consultado el 11 de octubre de 2026, desde https://www.yodaiken.com/papers/inherit.pdf

Zhang, X., Urban, C., & Wu, C. (2020). Priority inheritance protocol proved correct. *Journal of Automated Reasoning*, *64*(1), 73-95. https://doi.org/10.1007/s10817-019-09511-5